Terminagotchi en classe — fiches enseignant

Le terminal par le jeu, du cycle 3 au lycée · version de travail du 1er novembre 2026

Avant de commencer

Ce que c'est. Terminagotchi est un jeu où une créature se nourrit des vraies commandes tapées dans un terminal. L'histoire, « L'éveil du Terminaworld », conduit l'élève de son premier `pwd` jusqu'aux scripts, à git et au réseau, en 10 chapitres (le tronc commun), puis dans 8 mondes thématiques (réseaux, cybersécurité, cryptographie, matériel, radio, open source, IA, données).

Ce que ce n'est pas. Il n'y a ni compte, ni publicité, ni achat, ni chat. Le mode classe est **100 % local** : aucune donnée d'élève ne quitte l'ordinateur.

> Ces fiches sont une **version de travail**. Les durées, les questions et la correspondance avec les programmes doivent être relues par des enseignants et testées en classes pilotes avant de faire foi. Retours bienvenus.

Lancer le jeu sur les postes

- `python3 terminagotchi.pyz` : aucune installation (Python 3 suffit), sous Linux, macOS ou Windows. Le jeu fonctionne hors ligne.
- **Sous Windows**, l'élève ne tape pas dans PowerShell : le jeu ouvre **son propre terminal**, un petit Linux écrit en Python (aucun programme ajouté, rien d'installé). Dans la fenêtre du jeu, Entrée l'ouvre ; `tg terminal` ou la touche `[t]` en ouvre une fenêtre de plus. Les leçons et les commandes sont alors **les mêmes que sous Linux** (`ls`, `cd`, `chmod`, `./script.sh`, `for i in $(seq 20)` …), sur les vrais fichiers du terrain de jeu. Ce terminal n'écrit que dans `~/terrain_terminagotchi` et `~/.terminagotchi`, n'a pas accès à Internet et ne lance aucun programme de Windows, sauf le Python qui fait déjà tourner le jeu : `python3` (la Forge) lance pour de vrai les programmes de l'élève rangés dans le terrain, avec une garde qui refuse d'écrire ou de lire ailleurs, de lancer un autre programme ou d'aller sur le réseau ; `curl` (le Port) parle seulement au phare du jeu, sur l'ordinateur lui-même (`http://localhost:8042/`). Les quelques commandes qu'il n'a pas (réseau réel, `sudo`, compilateurs, `gpg`, `pip`, `python3 -m http.server`…) sont annoncées clairement et l'étape est racontée, comme sur le téléphone : l'élève continue sans être bloqué. Le jeu ouvre toujours ce terminal, même si WSL est installé sur le poste.
- Le jeu n'écrit que dans le dossier de l'élève : `~/terrain_terminagotchi` (le terrain de jeu, le seul endroit où l'on crée et supprime des fichiers) et `~/.terminagotchi` (la sauvegarde).
- Première séance : comptez 5 à 10 minutes pour l'éclosion de l'œuf (les premières commandes la déclenchent).
- Projection au tableau : `tg espace classe` (grand écran).

Le code de séance

L'enseignant donne un code ; l'élève tape `tg classe code <code> <pseudonyme>`, par exemple `tg classe code 05-GS-K Lynx42`.

Partie du code	Sens
`05`	le chapitre visé (`00` : pas de chapitre imposé, chacun continue où il en est)
`A`	sans aide : l'élève voit le but, pas la commande (cran C) ; `aide` ou 3 essais ratés la montrent
`G`	grand écran (le jeu rappelle `tg espace classe`)
`S`	sans chrono : les dressages de créatures n'ont plus de limite de temps
`K`	lettre de contrôle : une faute de frappe est détectée tout de suite

Pendant la séance : la faim de la créature est en pause, les notifications sont coupées, aucune synchronisation ni requête vers le site n'a lieu. `tg classe fin` termine la séance et rend les réglages d'avant.

Pour un autre réglage : `tg classe creer 5 grand sanschrono` (sur n'importe quel poste, hors ligne). La page « Pour les enseignants et les parents » du site donne le tableau de tous les codes.

Pseudonyme : 2 à 16 lettres ou chiffres, sans espace (« Prénom Nom » est refusé). Conseil : gardez la correspondance pseudonyme ↔ élève sur papier, en classe.

Le bilan et les crans

`tg classe bilan` affiche le bilan à l'écran ; `tg classe bilan fichier` l'enregistre en `.txt` (à imprimer) et en `.csv` dans le dossier personnel de l'élève, qui le remet lui-même (clé USB, dossier partagé, ENT). Côté enseignant, `tg classe rassembler <dossier>` réunit les `.csv` remis dans `bilan_de_classe.csv` et montre où la classe bloque. Les bilans sont rangés par date et par pseudonyme : **jamais de classement**.

Cran	Ce que l'élève a fait	Dans le bilan
A	réussi l'étape avec la commande montrée (découverte)	« étapes au cran A »
B	à trous (`ls -_`)	**pas encore dans le jeu** : le bilan ne l'invente pas
C	réussi l'étape en mode sans aide, commande cachée	« étapes au cran C », « commandes trouvées seul »
D	réussi un exercice de la salle d'entraînement (résultat vérifié)	« exercices vérifiés (cran D) »

Une notion est considérée comme **acquise au cran C ou D**. Le bilan liste aussi les étapes où l'élève a fait 3 essais ou plus sans avancer : c'est le point de départ de la reprise collective.

Reporté : un tableau de bord enseignant en ligne (classes créées sur un site, suivi à distance). Il demande un serveur, une décision du propriétaire du projet et un avis CNIL (données de mineurs). Il n'existe pas.

Organiser une séance type (55 minutes)

Temps	Phase
5 min	Lancement : `tg`, puis `tg classe code …` ; rappel du but du chapitre (au tableau, avec l'image du lieu)
30 min	Jeu en autonomie, en binômes si possible (un au clavier, un qui lit les explications) ; l'enseignant circule
10 min	Mise en commun : les questions pour l'oral, les blocages vus dans les bilans
5 min	Trace écrite (à coller dans le cahier)
5 min	`tg classe bilan fichier`, `tg classe fin`

Pour 45 minutes : réduisez le jeu à 25 minutes et la mise en commun à 7.

Fiche 1 — Le réveil : où suis-je ?

Objectif	se repérer dans l'arborescence : dossier courant, contenu d'un dossier, se déplacer, revenir à la maison, aller à la racine
Commandes	`pwd`, `ls`, `cd`, `cd ~`, `cd /`
Durée	55 min (éclosion de l'œuf comprise), ou 45 min si l'œuf a éclos avant
Prérequis	savoir lire une consigne courte, utiliser un clavier
Codes	guidée `01-GS-B` · sans aide `01-AGS-F` · postes seuls `01-S-E`

Déroulé. Lancement (5 min) ; éclosion de l'œuf et leçons « Où suis-je ? », « Le vieux chemin » (25 min) ; « Voyage jusqu'à la Racine » (10 min) ; mise en commun (10 min) ; trace écrite (5 min).

Questions pour l'oral.

- Que répond l'ordinateur quand on tape `pwd` ? Pourquoi le chemin commence-t-il par `/` ?
- Quelle différence entre « être dans un dossier » et « voir ce qu'il contient » ?
- Comment revenir chez soi, où que l'on soit ?

Trace écrite.

> Un ordinateur range ses fichiers dans des **dossiers**, comme les branches d'un arbre. Le tronc s'appelle la **racine** (`/`).
> `pwd` dit où je suis, `ls` montre ce qu'il y a ici, `cd lieu` me déplace, `cd ~` me ramène chez moi.

Sans ordinateur. L'arbre de la classe : dessiner au tableau un arbre de dossiers (école → classes → élèves) ; un élève « curseur » se déplace en lisant des cartes `cd …` ; la classe dit à chaque fois le résultat de `pwd`.

Fiche 2 — Le nid : créer, écrire, trouver ce qui est caché

Objectif	créer des dossiers et des fichiers, écrire dans un fichier, lire un fichier, voir les fichiers cachés
Commandes	`mkdir`, `touch`, `rmdir`, `rm -r`, `echo … >`, `cat`, `ls -a`, `cd ..`
Durée	55 min
Prérequis	fiche 1
Codes	guidée `02-GS-D` · sans aide `02-AGS-H` · postes seuls `02-S-G`

Déroulé. Rappel de la fiche 1 avec `pwd` et `ls` (5 min) ; « Construire mon nid », « Un lieu en trop » (15 min) ; « Le garde-manger », « Les passages secrets » (15 min) ; mise en commun (10 min) ; trace écrite (5 min).

Questions pour l'oral.

- Quelle différence entre un dossier et un fichier ? Comment le voit-on avec `ls` ?
- Que fait le signe `>` ? Que se passe-t-il si on l'utilise deux fois sur le même fichier ?
- Pourquoi certains fichiers sont-ils cachés ? Comment les reconnaît-on ?

Trace écrite.

> `mkdir` crée un dossier, `touch` crée un fichier vide, `echo texte > fichier` écrit dans un fichier (et remplace ce qu'il y avait), `cat fichier` le lit. Un nom qui commence par un point est **caché** : `ls -a` le montre. `rm` supprime **sans corbeille** : on vérifie avant.

Sans ordinateur. Boîtes et étiquettes : des boîtes (dossiers) et des cartes (fichiers) ; certaines cartes ont un point devant leur nom et restent face cachée tant qu'on n'a pas dit « ls -a ».

Fiche 3 — La forêt du Grenier : ranger

Objectif	voir toute une arborescence, déplacer, renommer, copier et supprimer des fichiers et des dossiers
Commandes	`ls -R`, `mv`, `cp`, `cp -r`, `rm`
Durée	45 min
Prérequis	fiches 1 et 2
Codes	guidée `03-GS-F` · sans aide `03-AGS-K` · postes seuls `03-S-J`

Déroulé. Rappel (5 min) ; « Le grand fouillis » (12 min) ; « Grand rangement » (13 min) ; mise en commun (10 min) ; trace écrite (5 min).

Questions pour l'oral.

- `mv` sert à deux choses : lesquelles ?
- Pourquoi faut-il `-r` pour copier un dossier ?
- Après `mv a.txt b.txt`, combien de fichiers y a-t-il ? Et après `cp a.txt b.txt` ?

Trace écrite.

> `mv source destination` **déplace** ou **renomme**. `cp source copie` **copie** (`cp -r` pour un dossier). `rm` supprime. `ls -R` montre tout ce qu'il y a dans les sous-dossiers.

Sans ordinateur. Le déménagement : par groupes, ranger un « grenier » de cartes en suivant une liste de commandes `mv` et `cp`, puis comparer les résultats entre groupes.

Fiche 4 — La caverne des Démons : qui a le droit ?

Objectif	savoir qui on est pour l'ordinateur, lire et modifier les droits d'un fichier, visiter (sans toucher) les lieux du système
Commandes	`whoami`, `sudo -h`, `ls -l`, `chmod 444`, `echo … >>`, `cat /etc/hostname`, `ls /bin`, `ls /var/log`
Durée	55 min
Prérequis	fiches 1 à 3
Codes	guidée `04-GS-H` · sans aide `04-AGS-N` · postes seuls `04-S-M`

Déroulé. Rappel (5 min) ; « Qui commande ici ? » (8 min) ; « Le bouclier » (12 min) ; « Les lieux du gardien » (10 min) ; mise en commun (12 min) ; trace écrite (5 min). Le jeu ne lance jamais `sudo` : `sudo -h` montre seulement l'aide.

Questions pour l'oral.

- Pourquoi l'ordinateur refuse-t-il d'écrire dans le fichier après `chmod 444` ?
- Que signifient les lettres `r`, `w`, `x` dans `ls -l` ?
- Pourquoi regarde-t-on `/etc` sans rien y modifier ?

Trace écrite.

> Chaque fichier a un **propriétaire** et des **droits** : lire (`r`), écrire (`w`), exécuter (`x`). `ls -l` les montre, `chmod` les change. `/etc` (réglages), `/bin` (programmes) et `/var/log` (journaux) appartiennent au système : on les regarde, on n'y touche pas.

Sans ordinateur. Les badges d'accès : chaque élève reçoit un badge (lire / écrire / exécuter) et ne peut faire que ce que son badge permet sur des « fichiers » affichés au tableau.

Fiche 5 — La bibliothèque engloutie : chercher

Objectif	trouver un fichier par son nom, lire un long texte, chercher un mot, compter des lignes, enchaîner deux commandes
Commandes	`find -name`, `less`, `grep`, `grep -rn`, `head -n`, `wc -l`, `|`
Durée	55 min
Prérequis	fiches 1 à 3
Codes	guidée `05-GS-K` · sans aide `05-AGS-Q` · postes seuls `05-S-P`

Déroulé. Rappel (5 min) ; « Le livre perdu » (12 min) ; « La loupe du détective » (18 min) ; mise en commun (12 min) ; trace écrite (5 min). Point d'attention : on sort de `less` avec la touche `q`.

Questions pour l'oral.

- Quelle différence entre `find` et `grep` ?
- Que fait le tuyau `|` dans `cat livre.txt | grep Abyssor` ?
- Comment savoir combien de lignes contient un texte sans le lire ?

Trace écrite.

> `find . -name "*mot*"` cherche des **fichiers** par leur nom. `grep mot fichier` cherche un **mot dans un texte**. `head` montre le début, `wc -l` compte les lignes. Le **tuyau** `|` donne le résultat d'une commande à la suivante.

Sans ordinateur. La chaîne de traitement : des élèves jouent les commandes (`cat`, `grep`, `wc`) et se passent une bande de papier qui se transforme à chaque étape.

Fiche 6 — La salle des machines du volcan : la place et la mémoire

Objectif	mesurer la mémoire et l'espace disque, trouver ce qui prend de la place, faire le ménage
Commandes	`free -h`, `du -sh`, `df -h`, `rm`
Durée	45 min
Prérequis	fiches 1 à 3
Codes	guidée `06-GS-N` · sans aide `06-AGS-S` · postes seuls `06-S-R`

Déroulé. Rappel (5 min) ; « Le monde étouffe » (12 min) ; « L'intendant » (10 min) ; mise en commun (12 min) ; trace écrite (5 min).

Questions pour l'oral.

- Quelle différence entre la mémoire vive et le disque ?
- Que veulent dire « K », « M », « G » dans les tailles ?
- Pourquoi l'ordinateur ralentit-il quand le disque est presque plein ?

Trace écrite.

> La **mémoire vive** sert pendant que l'ordinateur travaille (`free -h`), le **disque** garde les fichiers (`df -h`). `du -sh` dit combien pèse un dossier. Kilo, méga, giga : chaque unité vaut environ mille fois la précédente.

Sans ordinateur. Le bureau et l'armoire : le plan de travail (mémoire) est petit et rapide, l'armoire (disque) est grande et lente ; on simule un exposé à préparer avec des feuilles.

Fiche 7 — Le dortoir des brumes : les programmes qui tournent

Objectif	voir les programmes en cours, lancer une tâche en arrière-plan, l'arrêter
Commandes	`ps aux`, `sleep 300 &`, `jobs`, `kill %1`
Durée	45 min
Prérequis	fiches 1 à 3
Codes	guidée `07-GS-Q` · sans aide `07-AGS-U` · postes seuls `07-S-T`

Déroulé. Rappel (5 min) ; « Qui ne dort pas ? » (10 min) ; « Le chasseur » (12 min) ; mise en commun (12 min) ; trace écrite (5 min).

Questions pour l'oral.

- Combien de programmes tournent sur l'ordinateur en ce moment ? Pourquoi autant ?
- Que change le `&` à la fin d'une commande ?
- `kill` « tue »-t-il vraiment ? Que fait-il exactement ?

Trace écrite.

> Un programme qui tourne s'appelle un **processus** (`ps aux` les liste). Avec `&`, une commande travaille **en arrière-plan** pendant qu'on continue à taper. `jobs` liste mes tâches, `kill %1` arrête la première.

Sans ordinateur. Le chef d'orchestre : l'enseignant (le système) donne la parole à tour de rôle à des élèves « processus » ; certains travaillent « en arrière-plan » et lèvent la main quand ils ont fini.

Fiche 8 — La petite forge des Constructs : scripts et git

Objectif	écrire et lancer un petit script, en faire une commande, garder l'historique d'un travail avec git
Commandes	`sh`, `chmod +x`, `alias`, `git init`, `git status`, `git add`, `git commit`, `git diff`, `git log`
Durée	2 séances de 55 min (scripts, puis git)
Prérequis	fiches 1 à 4
Codes	guidée `08-GS-S` · sans aide `08-AGS-W` · postes seuls `08-S-V`

Déroulé. Séance 1 : « Le plan de travail », « Ma première recette », « Ma propre commande ». Séance 2 : « Mon premier dépôt », « Premier souvenir », « Voyage dans le temps ». Chaque séance : rappel (5 min), jeu (30 min), mise en commun (12 min), trace écrite (5 min). Si git n'est pas installé sur les postes, le jeu le dit.

Questions pour l'oral.

- Qu'est-ce qu'un script ? En quoi est-ce un programme ?
- Pourquoi faut-il `chmod +x` avant de lancer un script comme une commande ?
- À quoi sert un commit ? Que montre `git diff` ?

Trace écrite.

> Un **script** est un fichier qui contient des commandes : `sh script.sh` les exécute. Un **alias** donne un nom court à une commande. **git** garde des photos successives d'un travail (les **commits**) : on peut voir ce qui a changé (`git diff`) et l'historique (`git log`).

Sans ordinateur. La recette et l'album : écrire une recette pas à pas (script), puis photographier un dessin à chaque étape de sa réalisation (commits) et retrouver ce qui a changé entre deux photos.

Fiche 9 — La tour des nuages : le réseau

Objectif	tester une connexion, voir son adresse, demander une page web, comprendre le nom d'un site
Commandes	`ping -c 3`, `ip a`, `curl -I`, `curl -o`, `nslookup`, `ssh -V`, `traceroute`
Durée	55 min
Prérequis	fiches 1 à 3 et 5
Codes	guidée `09-GS-U` · sans aide `09-AGS-Y` · postes seuls `09-S-X`

Attention. Ce chapitre contacte vraiment `example.com` (un site réservé aux exemples). Vérifiez avant la séance que le filtrage de l'établissement laisse passer `ping` et `curl`. Sinon, préférez le monde 11 (le Port), où le réseau est **simulé** dans le jeu. Le mode classe ne bloque que les envois **du jeu** ; les commandes que l'élève tape fonctionnent normalement.

Déroulé. Rappel (5 min) ; « Le radar » (10 min) ; « Le facteur » (12 min) ; « Le cartographe » (10 min) ; mise en commun (12 min) ; trace écrite (5 min).

Questions pour l'oral.

- Que mesure `ping` ? Pourquoi le temps change-t-il d'une fois à l'autre ?
- Que se passe-t-il entre le moment où l'on tape une adresse et l'arrivée de la page ?
- À quoi sert `nslookup` ?

Trace écrite.

> Chaque machine du réseau a une **adresse IP** (`ip a`). `ping` vérifie qu'une machine répond, `curl` demande une page web, `nslookup` traduit un nom (example.com) en adresse : c'est l'annuaire d'Internet (**DNS**).

Sans ordinateur. Le facteur : des enveloppes avec une adresse IP, un annuaire nom → adresse, des élèves « routeurs » qui font passer les lettres de proche en proche.

Fiche 10 — Le cœur du monde : le grand défi

Objectif	réinvestir le tronc commun sur un problème complet, sans que les commandes soient données d'avance
Commandes	`rev`, `ls -a`, `find`, `cat … >`, `chmod 444`, `grep -rl … | wc -l`
Durée	55 min
Prérequis	au moins 5 des chapitres 3 à 9 (le jeu ouvre le chapitre 10 à ce moment-là)
Codes	**conseillé sans aide** `10-AGS-E` · guidée `10-GS-A` · postes seuls `10-S-D`

Déroulé. Présentation du défi au tableau (5 min) ; « Le message à l'envers », « Les morceaux du cœur », « Abyssor, le gardien » en binômes (35 min) ; mise en commun : chaque binôme explique une commande (10 min) ; bilan (5 min). C'est la séance où le bilan est le plus utile : les commandes trouvées seul (cran C) montrent ce qui est acquis.

Questions pour l'oral.

- Quelle commande vous a été la plus utile ? Pourquoi ?
- Comment avez-vous trouvé les fragments sans savoir où ils étaient ?
- Que feriez-vous autrement maintenant ?

Trace écrite.

> Pour résoudre un problème au terminal : **observer** (`ls -a`, `cat`), **chercher** (`find`, `grep`), **combiner** (`|`, `>`), **protéger** (`chmod`). Le terminal est une boîte à outils : on choisit l'outil selon le problème.

Sans ordinateur. L'escape game : une chasse au trésor en classe dont chaque indice est une commande à « exécuter » à la main (lire un papier caché, compter des mots, assembler des morceaux).

Les mondes du Carrefour (11 à 22) et le Sommet (23)

Après le tronc commun, chaque monde s'ouvre quand les chapitres nécessaires sont faits. Les fiches détaillées viendront après les classes pilotes ; en attendant, voici les codes et ce que chaque monde enseigne.

Monde	Ce qu'on y apprend	Guidée	Sans aide
11 · Le Port des messagers	réseaux et protocoles (TCP/IP, DNS, HTTP), dans un réseau **simulé**	`11-GS-C`	`11-AGS-G`
12 · La Citadelle des gardiens	cybersécurité défensive, boîtes d'entraînement entièrement simulées	`12-GS-E`	`12-AGS-J`
13 · La Crypte des Codes	chiffrement, empreintes, signatures, la blockchain expliquée de l'intérieur	`13-GS-G`	`13-AGS-M`
14 · La Salle des Rouages	matériel, système d'exploitation, machine virtuelle	`14-GS-J`	`14-AGS-P`
15 · La Tour des Ondes	ondes radio, spectre, LoRa, réseaux maillés (sans matériel)	`15-GS-M`	`15-AGS-R`
16 · La Guilde des Bâtisseurs Libres	logiciel libre, licences, contribuer avec git, fabrication	`16-GS-P`	`16-AGS-T`
17 · L'Observatoire de l'IA	comprendre l'IA et superviser des outils (aucune vraie IA dans le jeu)	`17-GS-R`	`17-AGS-V`
18 · La Salle des Données	texte, CSV, statistiques et SQL sur sa propre collection	`18-GS-T`	`18-AGS-X`
19 · La Forge	programmation : scripts, boucles, Python, son mini-jeu, son site servi sur sa machine, images et sons	`19-GS-V`	`19-AGS-Z`
20 · L'Atelier des robots	robotique simulée : capteurs et moteurs comme des fichiers, coordonnées, ET / OU / NON, la boucle lire-décider-agir, Raspberry Pi et embarqué racontés (aucun matériel)	`20-GS-B`	`20-AGS-F`
21 · La Bibliothèque du Temps	git : photographier ses fichiers, retrouver une version perdue, branches et fusion, empreintes ; tout reste sur la machine (aucun compte, rien n'est envoyé)	`21-GS-D`	`21-AGS-H`
22 · Le Poste de Contrôle	administration système **en lecture** : processus, mémoire et disque, variables d'environnement, journaux, cron, distributions, paquets, services, utilisateurs (rien n'est installé ni modifié)	`22-GS-F`	`22-AGS-K`
23 · Le Sommet	Expert++ (pour les plus grands) : scripts robustes, expressions régulières, débogage, documentation, défis ; s'ouvre quand trois mondes ont leur palier 3	`23-GS-H`	`23-AGS-N`

L'Atelier (chapitre 20) sous Windows. Le défi du robot est le même partout : dans le terminal du jeu, l'élève écrit

`cerveau.sh` et lance sa boucle `for i in $(seq 20); do sh cerveau.sh; atelier pas; done`, exactement comme sous Linux. Rien à régler sur les postes (pas de politique d'exécution des scripts à modifier).

Correspondance avec les programmes (indicative)

> À faire valider par des enseignants avant publication.

Niveau	Repères	Où dans le jeu
Cycle 3, collège (technologie, sciences numériques)	organiser des fichiers et des dossiers, chemins, premiers programmes	chapitres 1 à 4, 8
SNT (seconde)	Internet ; le Web ; les données structurées ; l'informatique embarquée	chapitre 9, monde 11 ; chapitre 5, monde 18 ; monde 15
NSI (première, terminale)	architecture matérielle et systèmes d'exploitation ; réseaux et routage ; sécurisation des communications	chapitres 4, 6, 7, monde 14 ; monde 11 ; mondes 12 et 13
CRCN / Pix	gérer des données ; environnements numériques ; protection et sécurité	chapitres 2 à 6 ; chapitre 9 ; monde 12

Données personnelles, en résumé

- Aucun compte, aucun nom, aucune adresse électronique : un pseudonyme.
- Rien n'est envoyé pendant les séances (synchronisation et requêtes du jeu coupées).
- Le bilan est un fichier sur l'ordinateur de l'élève ; c'est l'élève qui le remet.
- `tg classe effacer` oublie la séance ; supprimer `~/.terminagotchi` et `~/terrain_terminagotchi` efface tout le jeu du poste.
- Pas de tableau de bord en ligne : il est reporté (serveur, décision du propriétaire du projet, avis CNIL).